Studio Nx

Backend(FastAPI/Python) 62 — /api/v1/projects 라우터

2026-10-12

Backend(FastAPI/Python) 62 — 프로젝트 라우터 5개 (`/api/v1/projects`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	P1-13 프로젝트 API: 요청 주소 연결 (PRD 01 §7, FR-122~126)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	61 `delete_project` · (P1-13 끝: 개요 49, 다음 작업은 P1-14 myProject.html JS, 백엔드 다음 번호는 63)

1. 하는 일

요청 주소 5개를 서비스 함수(57~61)에 연결합니다. 라우터는 **처리 순서만 정하고**, 검사와 저장은 서비스가 맡습니다.

요청	성공	실패
`GET /api/v1/projects?query=&sort=&page=&size=`	200 `{items, total, page, size}`	로그인하지 않으면 401, 값이 틀리면 422 (page 0, size 51, 모르는 sort, 101자 검색어)
`POST /api/v1/projects`	**201** `ProjectItem` (OWNER, 멤버 1명, opened false)	401, 422 (56)
`GET /api/v1/projects/{id}`	200 `ProjectItem`	401, **404** (없는 번호, 남의 프로젝트, 삭제된 프로젝트)
`PATCH /api/v1/projects/{id}`	200 `ProjectItem` (바뀐 값)	401, 404, VIEWER는 **403**, 422
`DELETE /api/v1/projects/{id}`	**204** (본문 없음)	401, 404, OWNER가 아니면 **403**

응답 예 (`POST`, 본문 `{"title": "My Song", "bpm": 128.5}`):

```
{"project_id": 12, "title": "My Song", "description": null, "bpm": "128.50",
 "time_sig_num": 4, "time_sig_den": 4, "music_key": null,
 "created_at": "2026-10-08T02:10:11.123000", "updated_at": "2026-10-08T02:10:11.123000",
 "my_role": "OWNER", "member_count": 1, "opened": false}
```

`bpm`은 숫자가 아니라 **문자열**(`"128.50"`)로 옵니다. 소수(Decimal)를 정확하게 보내기 위한 pydantic의 규칙입니다. 화면(P1-14)에서 숫자가 필요하면 `Number(item.bpm)`으로 바꿔야 합니다.

2. 언제 쓰나

화면 (P1-14 myProject.html)	요청
페이지 열기, 검색, 정렬, 페이지 버튼	`GET /projects`
새 프로젝트 모달의 Create	`POST /projects` → 넓은 화면이면 시퀀서 창을 엽니다 (FR-126, FR-129)
카드의 Edit details에서 Save	`PATCH /projects/{id}`
카드의 Delete (+ `confirmDialog`)	`DELETE /projects/{id}`
시퀀서가 열릴 때 제목 표시 (Phase 2)	`GET /projects/{id}`

`CurrentUser`는 토큰을 검사해서 회원 정보를 넘겨주는 의존성입니다 (P1-03). 토큰이 없거나 만료되었다면 이 함수들은 호출되지도 않고 401로 응답합니다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Query, status

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.project import (
    PROJECT_PAGE_SIZE_MAX,
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
from app.services import project_service
```

`from app.services import project_service` 줄을 이번에 추가했습니다. 나머지는 템플릿 그대로이며, 주소와 요청, 응답 모양(`Query`, 스키마, `status`)을 정하는 데 씁니다.

함수 (설명 주석은 생략했습니다)

```
router = APIRouter()

NOT_FOUND = {404: {"model": ErrorResponse}}
FORBIDDEN = {403: {"model": ErrorResponse}}


@router.get("", response_model=ProjectListResponse)
def list_projects(
    user: CurrentUser,
    db: Db,
    query: Annotated[str, Query(max_length=100)] = "",
    sort: ProjectSort = ProjectSort.UPDATED,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=PROJECT_PAGE_SIZE_MAX)] = 10,
) -> ProjectListResponse:
    return project_service.list_projects(db, user, query.strip(), sort, page, size)


@router.post("", response_model=ProjectItem, status_code=status.HTTP_201_CREATED)
def create_project(data: ProjectCreateRequest, user: CurrentUser, db: Db) -> ProjectItem:
    project = project_service.create_project(db, user, data)
    # 방금 만든 프로젝트라 멤버는 나 하나 · 연 적 없음 → DB 에 다시 묻지 않음
    return project_service.to_item(db, project, project.members[0], member_count=1, opened=False)


@router.get("/{project_id}", response_model=ProjectItem, responses=NOT_FOUND)
```

```
def read_project(project_id: int, user: CurrentUser, db: Db) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    return project_service.to_item(db, project, member)


@router.patch(
    "/{project_id}",
    response_model=ProjectItem,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project(
    project_id: int,
    data: ProjectUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_project(db, project, member, data)
    return project_service.to_item(db, project, member)


@router.delete(
    "/{project_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def delete_project(project_id: int, user: CurrentUser, db: Db) -> None:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.delete_project(db, project, member)
```

문법 풀이

import 문법	뜻
`from app.services import project_service`	모듈 전체를 가져와서 `project_service.create_project`처럼 씁니다. 라우터 함수 이름(`create_project`)과 서비스 함수 이름이 같아도 서로 섞이지 않습니다

식	뜻
`@router.get("", response_model=...)`	이 함수가 `GET /api/v1/projects`를 처리합니다. 돌려준 값을 응답 스키마 모양으로 걸러서 JSON으로 보냅니다
`status_code=status.HTTP_201_CREATED`	성공 코드를 201(새로 만듦)로 정합니다. 삭제는 204(본문 없음)입니다
`responses={**NOT_FOUND, **FORBIDDEN}`	API 문서(/docs)에 404와 403 응답 모양을 적습니다. 동작에는 영향이 없습니다
`Annotated[int, Query(ge=1)] = 1`	주소의 `?page=` 값으로, 1 이상이어야 하고 없으면 1입니다. 틀리면 FastAPI가 422로 응답합니다
`query.strip()`	검색어 앞뒤 공백을 지웁니다. `" gam "`은 `"gam"`이 되고, `" "`은 `""`(전체 목록)이 됩니다
`project, member = ...get_membership(...)`	함수가 돌려준 두 값을 두 변수에 나눠 받습니다
`project.members[0]`	관계(P1-03)를 통해 이 프로젝트의 멤버 목록을 가져옵니다. 방금 넣은 OWNER 행 하나가 들어 있습니다

4. 흐름

요청	순서
목록	검색어 공백 제거 → `list_projects` (58)
만들기	`create_project` (59) → `to_item`에 멤버 1명, opened False를 넘김
상세	`get_membership` (57) → `to_item`
수정	`get_membership` → `update_project` (60) → `to_item`
삭제	`get_membership` → `delete_project` (61) → 204

어느 단계에서든 `AppError`가 나면 처리기(P1-03)가 `{"error": {"code", "message", "details"}}` 형태로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 연결만 함	권한, 검사, 저장은 서비스에 둡니다. 그래서 서비스는 HTTP 없이도 시험할 수 있고(57~61), 시퀀서 API(Phase 2)도 같은 함수를 쓸 수 있습니다
상세, 수정, 삭제 모두 `get_membership`부터	남의 프로젝트는 역할 검사 전에 404로 끝납니다 (57번의 결정 A)
만들기 응답에는 값을 직접 넘김	방금 만든 프로젝트의 멤버 수(1)와 opened(False)는 확실하므로 쿼리 두 번을 아낍니다 (템플릿)
수정과 상세도 `ProjectItem`으로 응답	목록 한 줄과 같은 모양이라서, 화면은 그 카드만 다시 그리면 됩니다
삭제는 204	돌려줄 내용이 없기 때문입니다. 화면은 성공하면 카드를 지우고 목록을 다시 받습니다

6. 시험 (`tests/test_project_routes.py`: 12개를 추가해서 모두 13개, 실제 앱과 MySQL로 실행)

시험	확인한 것
만들기	201, 응답 칸 12개, 제목 공백 정리, bpm `"128.50"`, 7/8, OWNER·1·false, DB의 주인 정보
만들기 422	공백 제목은 `VALIDATION_ERROR`입니다
로그인하지 않음	목록, 만들기, 상세 모두 401입니다
목록	이름순 2개씩이면 `alpha`, `Beta`이고 total은 3입니다. 검색어 `" gam "`으로 `Gamma`를 찾습니다
목록 422 (5가지)	page 0, size 0, size 51, sort random, 101자 검색어
상세	내 프로젝트는 200(만들기 응답과 같음)이고, 남의 프로젝트와 없는 번호는 404 "Project not found."입니다
수정	OWNER가 제목을 바꾸면 설명은 그대로이고 멤버는 3명입니다. EDITOR는 설명을 지울 수 있고, VIEWER는 403, 제목 null은 422이며, 이때 DB는 그대로입니다
삭제	EDITOR는 403이고, OWNER는 204(본문 없음)입니다. 그 뒤 상세 조회와 재삭제는 404이고, 목록의 total은 0입니다

전체 시험 492개가 통과했고, `ruff` 검사도 통과했습니다.